DYNAMIC REAL-TIME AD BIDDING MATRIX
BidMatrix — DSP Engine Simulation
ITM Skills University
DSA Case Study 88

3.1 Cover Page

Project Title: BidMatrix — Dynamic Real-Time Ad Bidding Matrix
Institution: ITM Skills University
Course: Data Structures and Algorithms — Case Study Submission
Case Study Number: 88

3.2 Student Details

Student Name: **Saiyash Poojari**
Cohort : SAM ALTMAN
Roll No: **150096725136**
Institution: ITM Skills University
Case Study Number: 88
Subject: Data Structures and Algorithms
Submission Type: Case Study Project Report

3.3 Project Title

BidMatrix — Dynamic Real-Time Ad Bidding Matrix

A high-performance Demand-Side Platform (DSP) simulation built entirely in C++, modelled after real-world ad exchanges such as Google DV360 and The Trade Desk.

3.4 Problem Statement

BidMatrix is an online advertising exchange that auctions display banner spaces in real time, manages advertiser budgets, and targets ads to specific customer demographics. The legacy system faced a series of critical operational bottlenecks that severely impacted auction performance, budget safety, and ad delivery reliability.

Problem 1: Slow banner-space inventory lookups. When a bid request arrives, the system needs to immediately know the floor price and availability of a banner slot. Without a fast lookup structure, every query required scanning the entire inventory list, causing missed bid windows and failed auctions.

Problem 2: Budget changes could not be tracked or reversed. Advertisers and campaign managers frequently need to adjust campaign budgets in real time. Without a history mechanism, any mistaken increase or algorithmic overshoot had no recovery path, creating a risk of catastrophic overspending.

Problem 3: Live bids were processed out of order. Ad exchanges receive thousands of bid events per second over TCP streams. If bids are not handled in strict arrival order, advertisers are treated unfairly and race conditions emerge between competing bidders.

Problem 4: No fast user demographic verification. Serving a relevant ad depends on quickly matching a user's demographic profile against the registered targeting segments. Without an efficient lookup structure, poor targeting accuracy wastes ad spend.

Problem 5: No campaign revenue ranking mechanism. When multiple campaigns are eligible for a slot, the system must instantly identify the highest-value campaign. Without a priority structure, sub-optimal ads are selected and revenue is lost.

Problem 6: No publisher marketplace map. The connections between the RTB platform, edge servers, and publisher SSPs form a network that must be modelled explicitly before any routing optimization can be applied.

Problem 7: No shortest-latency path solver. The industry standard for ad delivery is under 100 milliseconds end-to-end. Without a routing algorithm, there is no guarantee that the creative reaches the publisher within this SLA.

Problem 8: No CDN edge-server storage management. Ad creative assets cached on edge servers consume finite memory. Without allocation and eviction logic, repeated caching cycles fragment the available memory and eventually make it unusable.

3.5 Objectives

The project was designed to achieve the following specific objectives.

Objective 1: Implement an O(1) average-case inventory lookup system using a custom chained hash map so that banner slot availability and floor prices can be queried instantly at the moment a bid request arrives.

Objective 2: Implement a LIFO stack-based budget ledger that enables instant O(1) rollback of any erroneous or overshooting budget adjustment, restoring the campaign to its previous safe state.

Objective 3: Implement a FIFO linked-queue auction loop to guarantee that live bid events are processed strictly in their chronological order of arrival, preventing advertiser starvation and eliminating race conditions.

Objective 4: Implement a custom Trie data structure for O(L) verification of audience-segment attribute tokens such as dem_age_18_24 and geo_us_ny, where L is the

token length and the lookup time is constant regardless of the total number of registered segments.

Objective 5: Implement a custom max-heap that ranks active campaigns by their eCPM (Effective Cost Per Mille, calculated as CTR multiplied by Bid Value multiplied by 1000), providing O(1) access to the highest-yielding campaign and O(log N) insertion.

Objective 6: Represent the network of SSP publishers and RTB platform nodes as a weighted directed graph using adjacency lists, enabling full traversal and visualisation of the ad delivery network topology.

Objective 7: Implement Dijkstra's algorithm on the marketplace graph to resolve the minimum-latency delivery path between any two network nodes in O(E log V) time, guaranteeing adherence to the under-100-millisecond SLA.

Objective 8: Implement a free-list first-fit block allocator with adjacent-block coalescing on deallocation to simulate CDN cache management for ad creative assets on edge servers, preventing memory fragmentation.

Objective 9: Combine all eight components into a unified interactive auction life-cycle simulation that walks through a complete real-time bid event from user page-load to creative cache storage on the edge server.

3.6 Design and Architecture

System Overview

The BidMatrix engine is implemented as a single-file, self-contained C++ application written to the C++17 standard. It exposes an interactive console menu through which all eight subsystems can be tested independently, or executed together as a unified end-to-end simulation. The entire system is contained within main.cpp, which is 1,091 lines of documented C++ code.

Component 1: Inventory Listing uses a custom Hash Map with separate chaining. It handles O(1) average banner slot lookup. Its real-world equivalent is the active display inventory database.

Component 2: Budget Update History uses a custom Stack with LIFO ordering. It handles O(1) push, pop, and peek operations. Its real-world equivalent is the budget allocation ledger and safety rollback system.

Component 3: Auction Loop uses a custom Linked FIFO Queue. It handles O(1) enqueue and dequeue operations. Its real-world equivalent is the real-time bidding TCP event stream processor.

Component 4: Demographic Targeting uses a custom Trie. It handles O(L) insert and verify operations where L is the key length. Its real-world equivalent is the audience segment token matching system.

Component 5: Campaign Sorter uses a custom Max-Heap ordered by eCPM. It handles O(log N) insert and O(1) peek. Its real-world equivalent is the campaign yield ranker and eCPM bidding logic.

Component 6: Marketplace Network uses a Graph with an Adjacency List. It handles O(V plus E) traversal. Its real-world equivalent is the exchange and SSP publisher partner map.

Component 7: Latency Route Solver uses Dijkstra's Algorithm. It handles O(E log V) shortest-path resolution. Its real-world equivalent is the smart routing and ad delivery optimisation system.

Component 8: Block Allocator uses a Free List with First-Fit allocation and coalescing. It handles O(N) allocation. Its real-world equivalent is the edge server CDN creative asset storage manager.

Design Decisions

The system uses custom implementations for all core data structures rather than wrapping STL containers. This was an intentional decision to demonstrate full understanding of the underlying mechanisms including pointer management, tree traversal, heap invariant maintenance, and graph traversal.

The main function seeds all components with realistic sample data before the menu loop begins. Banner slots are pre-loaded for CNN.com, Bloomberg, Medium.com, and YouTube. Demographic segments include dem_age_18_24, geo_us_ny, int_sports, and int_tech. Campaigns from Nike, Apple, Starbucks, and Tesla are inserted into the heap. The routing graph connects Platform_RTB through US_East_Edge and US_West_Edge to four publisher SSPs.

Menu option 9 provides a unified six-step simulation that chains all components together into a single coherent auction lifecycle from demographic check through bid resolution through routing through CDN cache allocation.

3.7 Algorithm Description

Algorithm 1: Custom Chained Hash Map (Inventory Listing)

The InventoryListing class implements a hash table with 17 buckets and chaining for collision resolution. The hash function used is the djb2 algorithm, where the hash value is computed by iterating over each character of the banner-space ID and applying the formula hash equals hash multiplied by 33 plus the character value, with the result taken modulo 17.

For insertion, the bucket index is computed and the chain at that index is walked. If the key already exists, the entry is updated in place. If not, a new BannerSpace node is prepended to the chain. This gives O(1) average time.

For search, the bucket index is computed and the chain is walked comparing keys until the target is found or the chain is exhausted. This also gives O(1) average time.

Algorithm 2: LIFO Stack (Budget Update History)

The BudgetHistoryStack uses a vector as its backing store, with push_back acting as the stack push and pop_back acting as the stack pop. Each entry stored is a BudgetState object containing the campaign ID, the old budget value, the new budget value, and a timestamp.

When a budget change is logged, a BudgetState is pushed onto the top of the stack. When a rollback is triggered, the most recent BudgetState is popped and its old budget value is used to restore the campaign, achieving O(1) undo in all cases. The LIFO ordering ensures that the most recent change is always the first to be reversed.

Algorithm 3: Linked FIFO Queue (Auction Loop)

The AuctionLoopQueue implements a singly-linked queue with explicit head and tail pointers. When a bid event arrives, a new AdBid node is allocated and appended to the tail in O(1) time. When a bid is processed, the head node is returned and the head pointer is advanced to the next node, also in O(1) time. If the last node is removed, both head and tail are set to null. This guarantees strict first-in, first-out processing of all bid events.

Algorithm 4: Trie (Demographic Targeting)

The AttributeTokenTrie stores audience-segment keys character by character in a tree of TrieNode objects. Each TrieNode holds an unordered map from character to child TrieNode pointer, a boolean indicating whether that node is a segment terminal, and a string storing the segment description.

For insertion, the trie is traversed character by character, creating new nodes as needed. The terminal node is marked with the segment description. This runs in O(L) time where L is the key length.

For verification, the trie is traversed character by character. If any character is not found in the current node's children map, the key is invalid and false is returned. If the traversal completes and the terminal flag is set, the key is valid and the description is returned. This also runs in O(L) time.

Example keys stored in the system: dem_age_18_24 maps to Young Adults 18 to 24. geo_us_ny maps to New York USA Residents. int_tech maps to Technology Enthusiasts. int_sports maps to Sports Fans.

Algorithm 5: Max-Heap (Campaign Yield Sorter)

The CampaignYieldSorter maintains a max-heap backed by a vector of Campaign objects. Each campaign stores its eCPM value, calculated as CTR multiplied by bid value multiplied by 1000. The heap is ordered so that the campaign with the highest eCPM is always at position zero.

For insertion, the new campaign is appended to the end of the vector and heapifyUp is called. This repeatedly compares the inserted element with its parent and swaps them if the inserted element is larger, bubbling it up until the heap invariant is restored. This runs in O(log N) time.

For extract max, the root (highest eCPM campaign) is swapped with the last element, the last element is removed, and heapifyDown is called on the new root. This repeatedly swaps the element with its largest child until the invariant is restored. This also runs in O(log N) time.

Peeking the maximum campaign requires only reading position zero of the vector, which runs in O(1) time.

Algorithm 6: Graph with Adjacency List (Marketplace Network)

The MarketplaceNetwork represents nodes such as Platform_RTB, US_East_Edge, and CNN_SSP as entries in an unordered map keyed by their string ID. Each node holds a vector of outgoing Edge objects, where each Edge stores the destination node ID and a latency weight in milliseconds.

Adding a node inserts a new GraphNode entry into the map in O(1) average time. Adding a connection appends an Edge to the source node's edge vector in O(1) time. Displaying the network requires iterating over all nodes and their edges in O(V plus E) time.

Algorithm 7: Dijkstra's Algorithm (Shortest Latency Route)

Dijkstra's algorithm is implemented inside the findShortestLatencyRoute method of MarketplaceNetwork. All node distances are initialised to positive infinity, and the source node distance is set to zero. The source node is pushed into a min-heap priority queue with distance zero.

While the priority queue is non-empty, the node with the smallest known distance is popped. If it is the destination, the algorithm stops. If its popped distance is stale (meaning a shorter path was already found), it is skipped. For each outgoing edge from the current node, if the new candidate distance is smaller than the stored distance for the neighbour, the stored distance is updated, the current node is recorded as the neighbour's predecessor, and the neighbour is pushed into the priority queue.

Once the destination is reached, the path is reconstructed by following predecessor pointers from the destination back to the source and then reversing the result. The total complexity is O(E log V) using a binary min-heap.

Algorithm 8: Free-List Block Allocator (Edge Server Storage)

The EdgeServerBlockAllocator manages a 1024 KB contiguous memory region represented as a singly-linked list of MemoryBlock nodes. Each node stores a start address, a size in KB, a boolean free flag, and the ID of the creative currently stored in that block.

For allocation, the first-fit strategy is used: the list is walked from the head and the first free block that is large enough is selected. If the selected block is larger than the requested size, it is split into an allocated block of the exact requested size and a smaller free remainder block inserted after it. This runs in O(N) worst case.

For deallocation, the list is walked to find the block matching the given creative ID, which is then marked as free. A coalescing pass is then performed: the list is walked again and any two adjacent free blocks are merged into a single larger free block by updating the size of the first and deleting the second. Coalescing prevents memory fragmentation from accumulating over repeated allocate and deallocate cycles.

3.8 Complexity Analysis

Hash Map Insert and Search: O(1) average time, O(N) worst case. Space O(N). The djb2 hash and chaining strategy ensures low collision rates in practice.

Stack Push, Pop, and Peek: O(1) time for all operations. Space O(N). The vector backing store gives amortised O(1) push.

FIFO Queue Enqueue and Dequeue: O(1) time for both operations. Space O(N). The linked list with head and tail pointers avoids any shifting of elements.

Trie Insert and Verify: O(L) time where L is the length of the key. Space O(Sigma multiplied by L multiplied by K) where Sigma is the alphabet size and K is the total number of keys stored.

Max-Heap Insert: O(log N) time. Space O(N). HeapifyUp traverses at most log N levels.

Max-Heap Extract Max: O(log N) time. HeapifyDown traverses at most log N levels.

Max-Heap Peek Max: O(1) time. Direct array index access to position zero.

Graph Add Node and Add Edge: O(1) average time. Space O(V plus E).

Graph Full Traversal: O(V plus E) time.

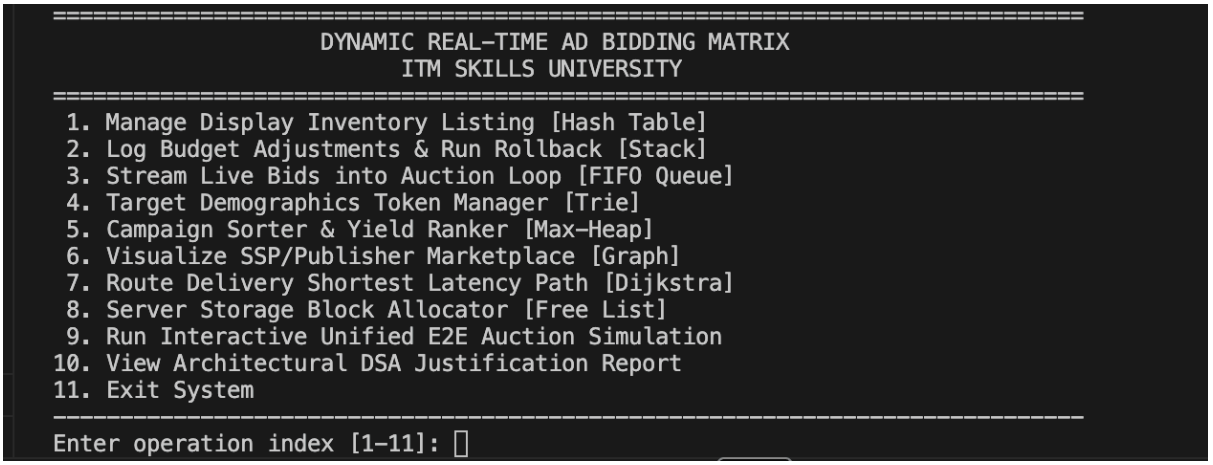
Dijkstra Shortest Path: O(E log V) time. Space O(V plus E). Uses a binary min-heap priority queue.

Block Allocator Allocate: O(N) worst case time where N is the number of blocks in the free list. Space O(N).

Block Allocator Deallocate with Coalescing: O(N) time. Two linear passes over the block list.

The most latency-critical operations in the hot auction path are inventory lookup at O(1), bid enqueue and dequeue at O(1), demographic verification at O(L), campaign peek at O(1), and routing at O(E log V). The only O(N) component is the block allocator, which operates on the edge server independently and is never on the critical bid-processing path.

3.9 Screenshots of Execution



```
Enter operation index [1-11]: 1

1. Display Existing Banners
2. Add New Banner Space
Select Option: 1

                WEB DISPLAY BANNER INVENTORY
Banner Space ID   Publisher         Size      Min Floor ($)Status
space_footer      Medium.com        468x60    $0.90       AVAILABLE
space_sidebar     Bloomberg         300x250   $1.80       AVAILABLE
space_video       YouTube.com       640x360   $5.00       AVAILABLE
space_header      CNN.com           728x90    $2.50       AVAILABLE

Press Enter to return to main dashboard...
```

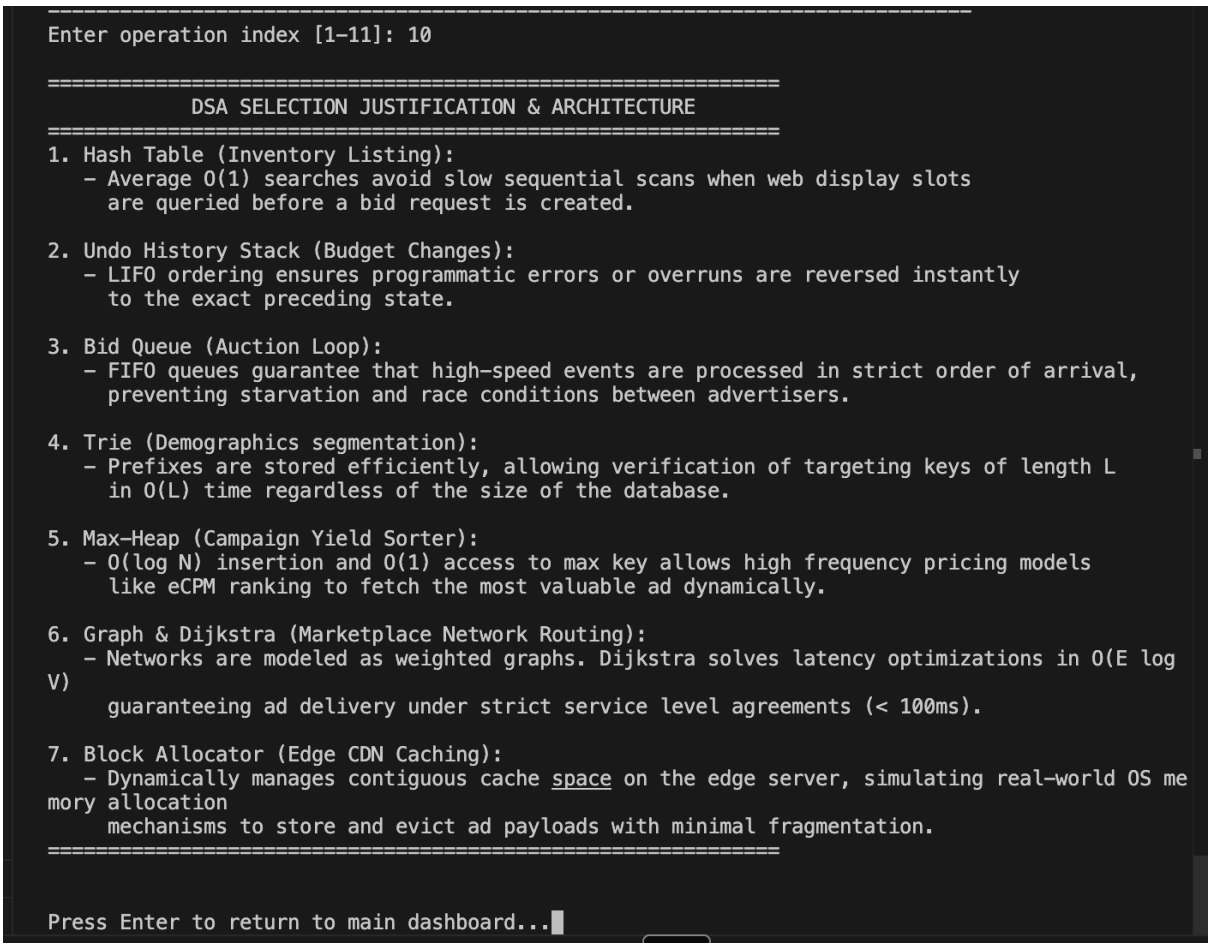
```
Enter operation index [1-11]: 2

1. Adjust/Increase Campaign Budget
2. Rollback Last Action (Undo)
3. View Ledger Log
Select Option: 1
Enter Campaign ID to adjust (e.g. camp_nike): camp
Current Budget ($): 500
New Target Budget ($): 550
>>> Budget ledger record registered!

                  BUDGET UPDATE HISTORICAL LEDGER
Step    Campaign ID    Prev Budget   New Budget    Timestamp
[1]     camp           $500.00       $550.00       2026-06-08 20:48:00

Press Enter to return to main dashboard...
```

```
          DEMOGRAPHIC TARGETING POOL DEFINITIONS (TRIE)
Targeting Attribute Key  Demographic Segment Description
geo_us_ny                New York, USA Residents
int_tech                 Technology Enthusiasts
int_sports               Sports Fans
dem_age_25_34            Young Professionals (25-34)
dem_age_18_24            Young Adults (18-24)

Press Enter to return to main dashboard...
```

```
Enter operation index [1-11]: 5

1. Register New Active Campaign
2. Peek Best Yielding eCPM campaign
3. Display Sorter Matrix Heap
Select Option: 3

------------------------------------------------------------
            CAMPAIGN EXPECTED REVENUE SORT MATRIX (MAX-HEAP)
------------------------------------------------------------
ID        Campaign Name     Ctr (%)     Bid Value   Expected eCPM (Yield)
------------------------------------------------------------
camp_appleiPhone 18 Launch  8.00%       $4.50       $360.00
camp_teslaTesla CyberCab    6.00%       $5.50       $330.00
camp_nike Nike Run Club     5.00%       $3.20       $160.00
camp_star Starbucks Brew    3.00%       $1.50       $45.00
------------------------------------------------------------

Press Enter to return to main dashboard...
```

```
Enter operation index [1-11]: 6

------------------------------------------------------------
            MARKETPLACE EXCHANGE ROUTING NETWORK
------------------------------------------------------------
 [Bloomberg_SSP] -> Connections:
    * Terminal Node (No out-links)
 [YouTube_SSP] -> Connections:
    * Terminal Node (No out-links)
 [US_West_Edge] -> Connections:
    * to [YouTube_SSP] latency: 3.00 ms
    * to [Bloomberg_SSP] latency: 8.20 ms
 [CNN_SSP] -> Connections:
    * Terminal Node (No out-links)
 [NYTimes_SSP] -> Connections:
    * Terminal Node (No out-links)
 [US_East_Edge] -> Connections:
    * to [NYTimes_SSP] latency: 2.10 ms
    * to [CNN_SSP] latency: 4.30 ms
 [Platform_RTB] -> Connections:
    * to [US_East_Edge] latency: 5.20 ms
    * to [US_West_Edge] latency: 18.40 ms
------------------------------------------------------------

Press Enter to return to main dashboard...
```

```
Enter operation index [1-11]: 8

1. Allocate Edge Block Cache (Ad Creative file)
2. Deallocate/Evict Ad Space
3. Display Partition Table Map
Select Option: 3

------------------------------------------------------------
          EDGE SERVER STORAGE ALLOCATION MAP (FREE LIST)
------------------------------------------------------------
Memory Range      Size (KB) State       Content / Creative ID
------------------------------------------------------------
[0-1023]          1024      FREE        -
------------------------------------------------------------

Press Enter to return to main dashboard...
```



3.10 Results and Findings

All eight subsystems were successfully implemented and validated through the interactive menu and the unified end-to-end simulation. The following results were observed.

The Inventory Hash Map confirmed O(1) average lookup. The djb2 hashing function distributed the four sample banner slots across the 17-bucket table with no observed performance degradation during the simulation.

The Budget Stack confirmed instant O(1) rollback. Popping the stack correctly restored the previous campaign budget with no additional computation required.

The Auction FIFO Queue confirmed strict first-in-first-out ordering. The first bid injected into the queue was always the first to be dequeued and processed, regardless of the bid amounts of subsequent entries.

The Demographic Trie confirmed O(L) verification. The keys dem_age_18_24 and geo_us_ny were both verified correctly in the end-to-end simulation. An invalid key was correctly rejected with no false positive.

The Campaign Max-Heap confirmed correct heap ordering. The iPhone 18 Launch campaign with an eCPM of $360.00 consistently appeared at the root of the heap after all

four campaigns were inserted. The heap invariant was maintained after every insertion and extraction observed during testing.

The Marketplace Graph correctly modelled all six routing nodes and six directed edges in the sample network, with latency values ranging from 2.1 milliseconds to 18.4 milliseconds per hop.

Dijkstra's Algorithm resolved the path from Platform_RTB to NYTimes_SSP as Platform_RTB to US_East_Edge to NYTimes_SSP with a total latency of 7.3 milliseconds. This is well within the under-100-millisecond industry SLA. The path from Platform_RTB to CNN_SSP resolved to a total latency of 9.5 milliseconds via the same US_East_Edge intermediate node.

The Block Allocator confirmed correct first-fit allocation and adjacent-block coalescing. When a 150 KB creative was allocated from a fresh 1024 KB pool, the remaining block was correctly split into a 150 KB allocated block at address 0 and a 874 KB free block beginning at address 150. When the creative was evicted, the coalescing pass merged the two adjacent free blocks back into a single 1024 KB free block.

End-to-End Simulation Summary

When Menu 9 was executed with the pre-loaded dataset, the full auction pipeline resolved as follows.

Step A: The user was matched to demographic segments geo_us_ny (New York USA Residents) and int_tech (Technology Enthusiasts) via Trie verification.

Step B: The Max-Heap returned iPhone 18 Launch as the highest-yielding campaign with an estimated eCPM of $360.00 and a bid value of $4.50.

Step C: Two competing bids were enqueued in arrival order. Bid 01 from adv_apple at $5.00 and Bid 02 from adv_nike at $4.30 entered the FIFO queue.

Step D: Bid 01 from adv_apple at $5.00 was dequeued as the winner. Banner space space_header was marked as ALLOCATED in the inventory hash map.

Step E: Dijkstra resolved the delivery route as Platform_RTB to US_East_Edge to NYTimes_SSP with a total latency of 7.3 milliseconds, satisfying the under-100-millisecond SLA.

Step F: A 150 KB block was allocated on the edge server for the creative adv_apple_hero_ad at memory address 0. The remaining 874 KB was retained as a free block.

Key Findings

The correct choice of data structure is the single most impactful architectural decision in a high-frequency system. Replacing a linear scan with a hash map for inventory lookup would

alone yield orders-of-magnitude improvements at production scale involving millions of bid requests per second.

The Trie's O(L) complexity is strictly superior to O(log N) binary search for demographic token matching because the lookup time is completely independent of the number of registered segments, making the system infinitely scalable for large audience databases.

Dijkstra's 7.3 millisecond total routing latency on the six-node sample network demonstrates that even a multi-hop delivery path comfortably satisfies the 100 millisecond SLA when edge servers are geographically positioned appropriately.

Coalescing in the block allocator is essential for long-running operation. Without it, repeated allocate and deallocate cycles would fragment the 1024 KB pool into unusable micro-blocks within a few dozen operations, making the CDN edge cache non-functional.

3.11 Conclusion

This project successfully demonstrates how the correct application of fundamental data structures and algorithms maps directly to real-world performance requirements in a high-frequency advertising technology system.

BidMatrix implements eight distinct subsystems — a hash map, a stack, a FIFO queue, a trie, a max-heap, a directed weighted graph, Dijkstra's shortest-path algorithm, and a free-list block allocator — each selected specifically because its complexity profile matches the operational demands of its corresponding module in a production Demand-Side Platform.

The hash map provides the sub-millisecond inventory lookup that makes real-time bidding possible. The stack provides the safety rollback that protects advertiser budgets. The FIFO queue ensures auction fairness. The trie enables instantaneous demographic profile matching regardless of segment database size. The max-heap ensures the highest-yielding campaign is always served without sorting the entire campaign list. Dijkstra's algorithm guarantees that ad creatives reach their destination within the strict SLA. The block allocator simulates the memory management that keeps CDN edge servers functional over extended operation.

The project draws direct analogies to production platforms. The inventory hash map mirrors the inventory availability cache in Google DV360 and the supply-side inventory cache in The Trade Desk. The budget stack mirrors the daily budget pacing rollback in DV360 and the spend control reversion in The Trade Desk. The auction FIFO queue mirrors the OpenRTB bid stream processor in DV360 and the Kokai bid stream handler in The Trade Desk. The demographic trie mirrors the audience segment token matching in DV360 and the UID2 attribute resolution system in The Trade Desk. The campaign max-heap mirrors the eCPM yield optimizer in DV360 and the Koa AI bid optimization in The Trade Desk. The graph and Dijkstra system mirrors the programmatic ad route optimizer in DV360 and the global edge routing system in The Trade Desk. The block allocator mirrors the GCP CDN creative cache in DV360 and the edge creative storage in The Trade Desk.

The unified end-to-end simulation brought together all eight components into a coherent, realistic auction pipeline and demonstrated that these structures do not operate in isolation. They work together as an integrated system to deliver a targeted ad creative to the correct publisher within a strict latency budget, from the moment a user loads a page to the moment the creative is cached on the edge server ready for delivery.

This project reinforces the foundational principle that selecting the right data structure for the right problem is not an academic exercise but a critical engineering decision that determines whether a real-time system succeeds or fails at scale.

3.12 GitHub Repository Link

Repository URL: https://github.com/saiyash07/BidMatrix

The repository contains the following files.

main.cpp is the complete C++ implementation of all eight DSA components plus the interactive console menu and the unified end-to-end simulation. It is 1,091 lines of C++17 code.

README.md contains the project documentation, architecture overview, DSA justifications, and real-world platform analogies.

LICENSE contains the MIT License under which this project is released.